

```

1: // C Program to implement Linear Search using Iteration
2: #include <stdio.h>
3: int linearSearch(int arr[], int size, int target)
4: {
5:     for (int i = 0; i < size; i++)
6:     {
7:         if (arr[i] == target)
8:         {
9:             return i; // Return the index if target is found
10:        }
11:    }
12:    return -1; // Return -1 if target is not found
13: }
14: int main()
15: {
16:     int arr[] = {5, 9, 2, 7, 1};
17:     printf("\nsizeof(arr[0])=%d,sizeof(arr)=%d\n",sizeof(arr[0]),
sizeof(arr));
18:     int size = sizeof(arr) / sizeof(arr[0]);
19:     int target = 7;
20:
21:     int result = linearSearch(arr, size, target);
22:
23:     if (result != -1)
24:     {
25:         printf("Element %d found at index %d.\n", target, result);
26:     }
27:     else
28:     {
29:         printf("Element %d not found in the array.\n", target);
30:     }
31:
32:     return 0;
33: }
34:
35: // C Program to implement Linear Search using Recursion
36: #include <stdio.h>
37: int linearSearchRecursive(int arr[], int start, int end, int target)
38: {
39:     if (start > end)
40:     {
41:         return -1; // Base case: target element not found
42:     }
43:
44:     if (arr[start] == target)
45:     {
46:         return start; // Base case: target element found at index start
47:     }
48:

```

```

49:     return linearSearchRecursive(arr, start + 1, end, target); //
    Recursive call
50: }
51: int linearSearch(int arr[], int size, int target)
52: {
53:     return linearSearchRecursive(arr, 0, size - 1, target);
54: }
55: int main()
56: {
57:     int arr[] = {5, 9, 2, 7, 1};
58:     int size = sizeof(arr) / sizeof(arr[0]);
59:     int target = 7;
60:
61:     int result = linearSearch(arr, size, target);
62:
63:     if (result != -1)
64:     {
65:         printf("Element %d found at index %d.\n", target, result);
66:     }
67:     else
68:     {
69:         printf("Element %d not found in the array.\n", target);
70:     }
71:     return 0;
72: }
73:
74: //A non-recursive program for binary search:
75: # include <stdio.h>
76: void main()
77: {
78:     int number[25], n, data, i, flag = 0, low, high, mid;
79:     printf("\n Enter the number of elements: ");
80:     scanf("%d", &n);
81:     printf("\n Enter the elements in ascending order: ");
82:     for(i = 0; i < n; i++)
83:         scanf("%d", &number[i]);
84:     printf("\n Enter the element to be searched: ");
85:     scanf("%d", &data);
86:     low = 0; high = n-1;
87:     while(low <= high)
88:     {
89:         mid = (low + high)/2;
90:         if(number[mid] == data)
91:         {
92:             flag = 1;
93:             break;
94:         }
95:         else
96:         {

```

```

97:         if(data < number[mid])
98:             high = mid - 1;
99:         else
100:             low = mid + 1;
101:     }
102: }
103: if(flag == 1)
104:     printf("\n Data found at location: %d", mid + 1);
105: else
106:     printf("\n Data Not Found ");
107: }
108:
109: //A recursive program for binary search:
110: # include <stdio.h>
111: void bin_search(int a[], int data, int low, int high)
112: {
113:     int mid ;
114:     if( low <= high)
115:     {
116:         mid = (low + high)/2;
117:         if(a[mid] == data)
118:             printf("\n Element found at location: %d ", mid + 1);
119:         else
120:         {
121:             if(data < a[mid])
122:                 bin_search(a, data, low, mid-1);
123:             else
124:                 bin_search(a, data, mid+1, high);
125:         }
126:     }
127:     else
128:         printf("\n Element not found");
129: }
130: void main()
131: {
132:     int a[25], i, n, data;
133:     printf("\n Enter the number of elements: ");
134:     scanf("%d", &n);
135:     printf("\n Enter the elements in ascending order: ");
136:     for(i = 0; i < n; i++)
137:         scanf("%d", &a[i]);
138:     printf("\n Enter the element to be searched: ");
139:     scanf("%d", &data);
140:     bin_search(a, data, 0, n-1);
141: }

```